



Techniques of Compile-time Unit Testing in C++

Braden Ganetsky

Thank you!

Braden Ganetsky

braden@ganets.ky

GitHub @k3DW

<https://ganets.ky/slides/2026/accuonsea>



**I've been very interested in
compile-time testing recently...**



Goals of this talk

Need

- Test features at compile-time and run-time
- Helpful error messages at run-time

Want

- Write test only once, compile-time and run-time together
- Equally helpful error messages from compile-time cases

Compile-time unit testing - Introduction

Braden Ganetsky - C++Now 2024

C++now 2024
Aspen, Colorado

CppNow.org

Video Sponsorship Provided By

millennium
think-cell



Unit Testing an Expression Template Library in C++20

Braden Ganetsky

"Unit Testing an Expression Template Library in C++20"

<https://www.youtube.com/watch?v=H4KzM-wDiQw>



Goals of this talk

Need

- **Test features at compile-time and run-time**
- **Helpful error messages at run-time**

Want

- **Write test only once, compile-time and run-time together**
- **Equally helpful error messages from compile-time cases**





"Testing Everything in Constexpr"



My claim

- It is good engineering practice to test the code you write
- Corollary: It is *bad* engineering practice to *not* test the code you write



My claim

- I have library that I claim is fully capable at compile-time...
- ...then it should be tested at compile-time
- I write compile-time code, therefore I should test it



Divergent behaviour

```
int main() {  
    {  
        any_of_parser<"abc"> p;  
        static_assert(p.parse("b"));  
        assert(p.parse("b"));  
    }  
}
```

```
int main() {  
    [] {  
        any_of_parser<"abc"> p;  
        static_assert(p.parse("b"));  
        assert(p.parse("b"));  
        // Assertion failed: p.parse("b")  
    }();  
}
```

<https://godbolt.org/z/nhTfbvPnz>



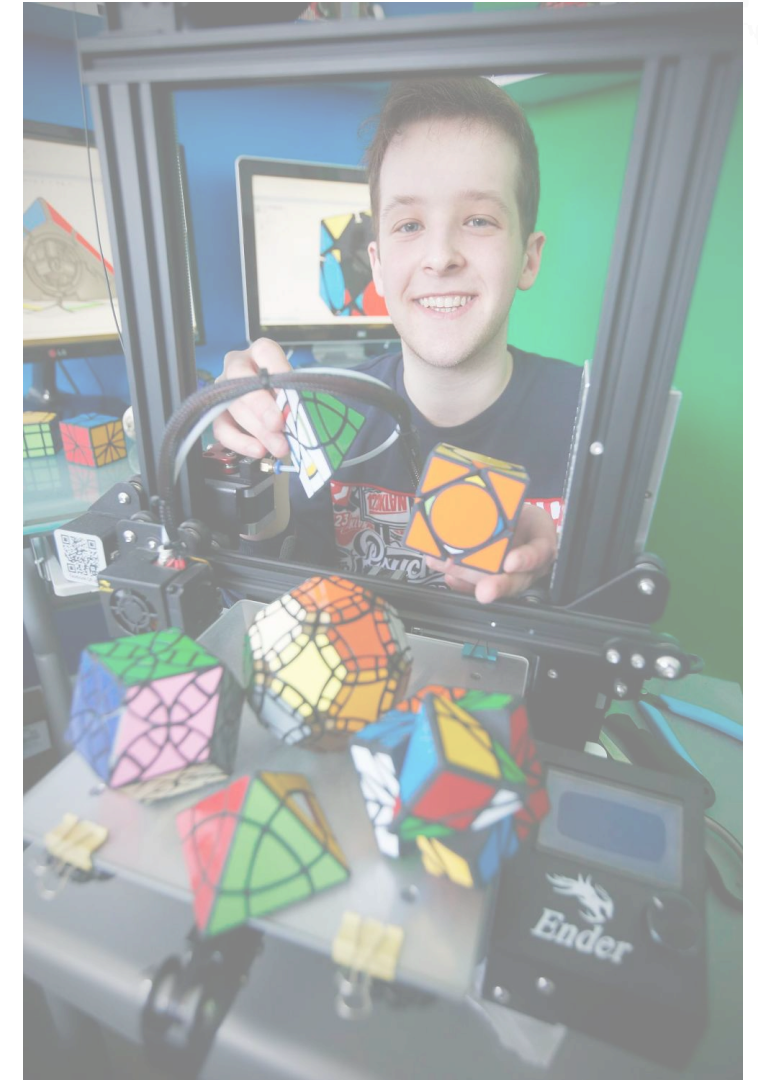
This talk is about

- Decisions and methods I used in writing a library
- Testing the correctness of your program...
- For code that runs at compile-time...
- In a way that is ergonomic enough to remove friction



About me

- Canadian
- Work at LSEG
- Secretary of WG21
- Facilitator of Winnipeg C++ Developers
- Licensed professional engineer
- I like twisty puzzles



Overview

- Compile-time testing design decisions
- My compile-time-first unit testing library `k3tchup`
- "Regular" checks in `k3tchup`
- "Stateful" checks in `k3tchup`
- Questions at any time!



Compile-time testing design decisions

Compile-time testing design decisions

- "Out-of-source" vs "in-source"
- When to report the errors
- Test runner
- Early exit mechanism



Out-of-source compile-time testing

```
template <class T>
struct wrapper {
    wrapper(T value) :
        _value(std::move(value)) {}

    const T& operator()() const {
        return _value;
    }

private:
    T _value;
};
```

```
#include "file.hpp"
int main() {
    wrapper<int> w{42};
    w() = 67;
}
```

```
add_executable(compile_fail_test EXCLUDE_FROM_ALL
    file_test.cpp
)

target_include_directories(compile_fail_test
    PRIVATE .
)

add_test(
    NAME compile_fail_test
    COMMAND ${CMAKE_COMMAND}
        --build ${CMAKE_BINARY_DIR}
        --target compile_fail_test
)

set_tests_properties(compile_fail_test
    PROPERTIES
        WILL_FAIL TRUE
)
```



Out-of-source compile-time testing

```
$ ctest -C Debug
Test project path/to/build
  Start 1: compile_fail_test
1/1 Test #1: compile_fail_test ..... Passed    0.51 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =  0.51 sec
```

```
$ cmake --build build --target compile_fail_test
...
path\to\file_test.cpp(4,6):
  error C3892: 'x': you cannot assign to a variable that is const
```



Out-of-source compile-time testing

```
template <class T>
struct wrapper {
    wrapper(T value) :
        _value(std::move(value)) {}

    const T& operator()() const {
        return _value;
    }

private:
    T _value;
};
```

```
#include "file.hpp"
int main() {
    wrapper<int> w{42};
    w() = 67;
}
```

```
add_executable(compile_fail_test EXCLUDE_FROM_ALL
    file_test.cpp
)

target_include_directories(compile_fail_test
    PRIVATE .
)

add_test(
    NAME compile_fail_test
    COMMAND ${CMAKE_COMMAND}
        --build ${CMAKE_BINARY_DIR}
        --target compile_fail_test
)

set_tests_properties(compile_fail_test
    PROPERTIES
        WILL_FAIL TRUE
)
```



Out-of-source compile-time testing

```
$ ctest -C Debug
Test project path/to/build
  Start 1: compile_fail_test
1/1 Test #1: compile_fail_test ..... Passed    0.51 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =  0.51 sec
```

```
$ cmake --build build --target compile_fail_test
...
path\to\file_test.cpp(1,1):
  error C1083: Cannot open include file: 'file.hpp': No such file or directory
```



Out-of-source testing: pros

- No extra language support needed
- Test absolutely anything, no matter how complex



Out-of-source testing: cons

- Only one granular check per file
- Slower iteration time, must reconfigure for every new check
- False positive, *fails for the wrong reason*
- Doesn't tell you *why* the failure occurred



In this case...

- Checking correct file inclusion
- Checking syntax in the whole file
- Checking semantics of `wrapper<int> w{42};`
- Checking semantics of `w() = 67;`
- ...and this is a small file



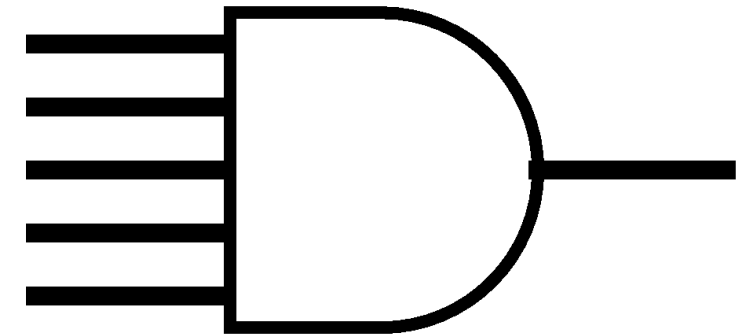
Aside: expect-success tests

- Every condition must succeed
- Then the whole test passes
- There is only *one* way the test can pass



Aside: expect-success tests

Check 1	Check 2	Whole Test
✗	✗	✗
✗	✓	✗
✓	✗	✗
✓	✓	✓



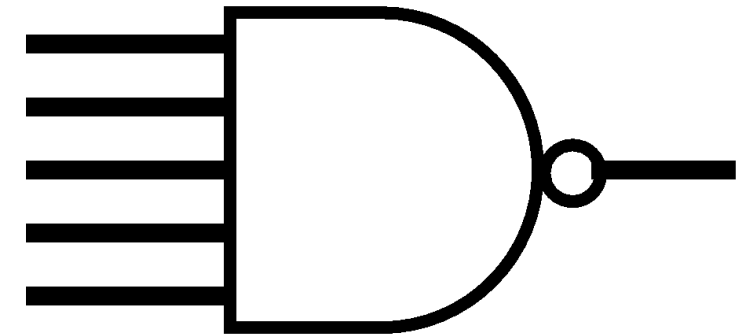
Aside: expect-failure tests

- Some condition must fail
- Then the whole test passes
- There are many ways the test can pass



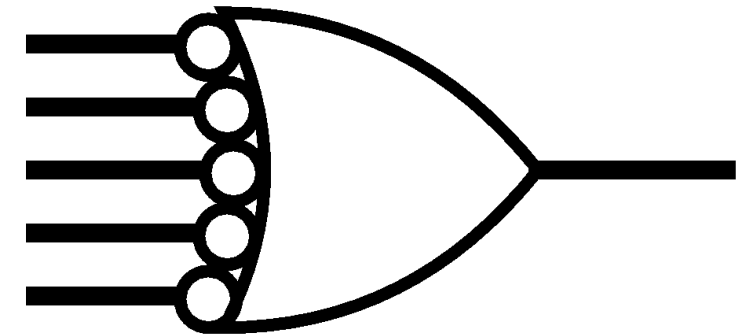
Aside: expect-failure tests

Check 1	Check 2	Whole Test
✗	✗	✓
✗	✓	✓
✓	✗	✓
✓	✓	✗



Aside: expect-failure tests

Check 1	Check 2	Whole Test
✗	✗	✓
✗	✓	✓
✓	✗	✓
✓	✓	✗



Aside: expect-failure tests

- Many ways the test can pass
- Only *one* way the test can fail
- This is brittle



Compile-time testing design decisions

1. Use expect-success testing



```

template <class T>
struct wrapper {
    wrapper(T value) :
        _value(std::move(value)) {}

    const T& operator>()() const {
        return _value;
    }

private:
    T _value;
};

```

```

#include "file.hpp"
int main() {
    wrapper<int> w{42};
    static_assert(not is_assignable_v<
        decltype(w()), int>);
}

```

```

add_executable(wrapper_test
    EXCLUDE_FROM_ALL
    file_test.cpp
)

target_include_directories(
    wrapper_test PRIVATE .
)

add_test(
    NAME wrapper_test
    COMMAND ${CMAKE_COMMAND}
        --build ${CMAKE_BINARY_DIR}
        --target wrapper_test
)

```



In-source compile-time testing

- Either `static_assert` / other compile-time failures,
- or run-time failures
- ...with detection idiom, type traits, concepts, etc
- ...with `constexpr` code



Detection idiom, type traits, concepts

- Expect-success testing
- Properties necessarily known at compile-time
- No point in run-time checking



Detection idiom, type traits, concepts

```
template <class T>
struct wrapper {
    wrapper(T value) :
        _value(std::move(value)) {}

    const T& operator()() const {
        return _value;
    }

private:
    T _value;
};
```

```
#include "file.hpp"
int main() {
    wrapper<int> w{42};
    static_assert(not is_assignable_v<decltype(w()), int>);
    assert(not is_assignable_v<decltype(w()), int>);
}
```



constexpr code

- User code executed at compile-time
- ...therefore much more complexity
- Possible that run-time code will differ
- No UB in `constexpr` code allowed



When to report the errors

1. Report the compile-time error at compile-time
2. Report the run-time error at run-time
3. ??





When to report the errors

```
constexpr auto abc = "abc"_all;  
  
constexpr auto result = abc.parse("abcd");  
  
static_assert(result.has_value());  
static_assert(result.value() == "abc");  
static_assert(result.remaining() == "d");
```

```
auto abc = "abc"_all;  
  
auto result = abc.parse("abcd");  
  
assert(result.has_value());  
assert(result.value() == "abc");  
assert(result.remaining() == "d");
```



When to report the errors

```
constexpr auto abc = "abc"_all;  
constexpr auto result = abc.parse("abcd");
```

```
constexpr bool b1 = result.has_value();  
assert(b1);
```

```
constexpr bool b2 = result.value() == "abc";  
assert(b2);
```

```
constexpr bool b3 = result.remaining() == "d";  
assert(b3);
```



When to report the errors

- We can store a condition at compile-time
- ...and then check it at run-time



Goals of this talk

Need

- Test features at compile-time and run-time
- Helpful error messages at run-time

Want

- Write test only once, compile-time and run-time together
- Equally helpful error messages from compile-time cases

Compile-time unit testing - Introduction

Braden Ganetsky - C++Now 2024

C++**now** 2024
Aspen, Colorado

CppNow.org

Video Sponsorship Provided By

millennium
think-cell



Unit Testing an Expression Template Library in C++20

Braden Ganetsky

"Unit Testing an Expression Template Library in C++20"

<https://www.youtube.com/watch?v=H4KzM-wDiQw>

Aside: snitch library

- <https://github.com/snitch-org/snitch>
- Changed the way I think about compile-time testing



Aside: snitch library

```
#include <snitch/snitch.hpp>

constexpr unsigned int Factorial( unsigned int number ) {
    return number <= 1 ? number : Factorial(number-1)*number;
}

TEST_CASE("Factorials are computed", "[factorial]") {
    CONSTEVAL_CHECK(Factorial(0) == 1u);
    CONSTEXPR_CHECK(Factorial(5) == 6u);
}
```

Does the compile succeed or fail?



Aside: snitch library

```
> path\to\build\snitch_example.exe
```

```
failed: running test case "Factorials are computed"
```

```
    at path\to\snitch_example.cpp:8
```

```
    CONSTEVAL_CHECK(Factorial(0) == 1u), got: 0 != 1
```

```
failed: running test case "Factorials are computed"
```

```
    at path\to\snitch_example.cpp:9
```

```
    CONSTEXPR_CHECK[compile-time](Factorial(5) == 6u), got: 120 != 6
```

```
failed: running test case "Factorials are computed"
```

```
    at path\to\snitch_example.cpp:9
```

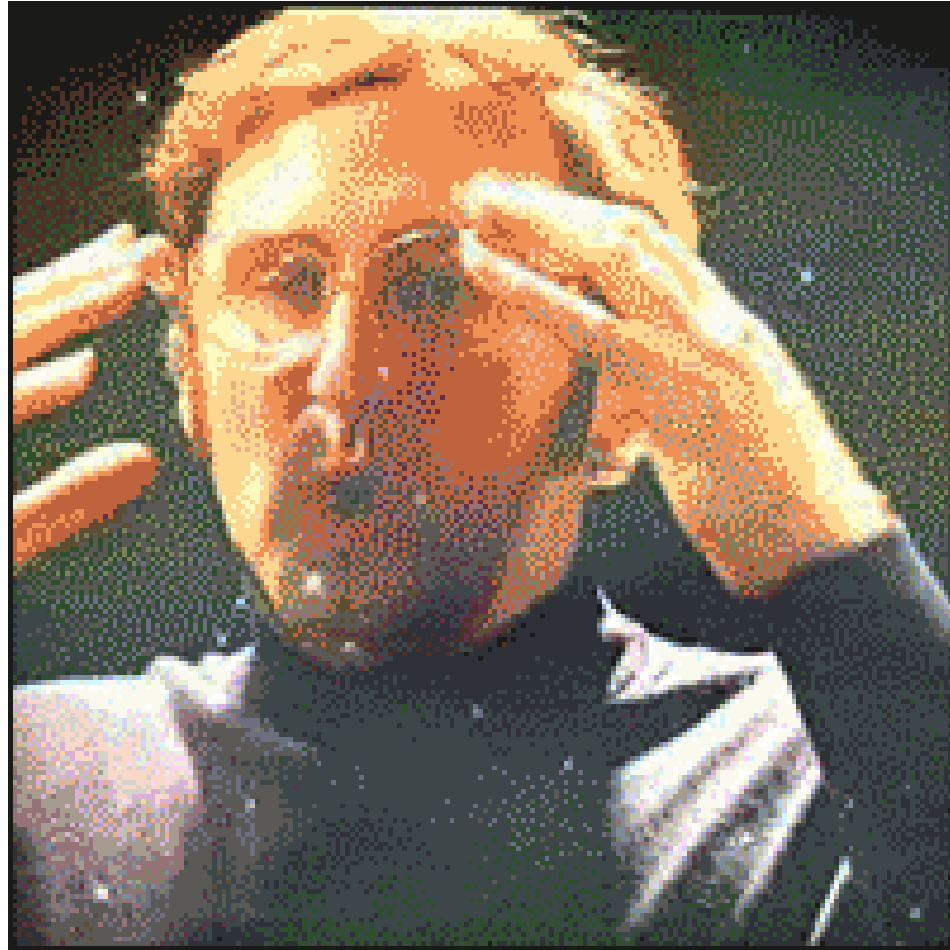
```
    CONSTEXPR_CHECK[run-time](Factorial(5) == 6u), got: 120 != 6
```



Aside: snitch library

- <https://github.com/snitch-org/snitch>
- **Compile-time checks don't fail the compile!**
- **Even with a compile-time check, we get a nice user-defined printout**





Compile-time testing design decisions

1. Use expect-success testing
2. Report compile-time errors at run-time



Test runner

- A test runner only makes sense at run-time
- Otherwise we just have compile-time failures
- ... therefore all code must compile, even the compile-time errors



With a full-fledged run-time test runner:

- Filtering of the tests
- Repetition of the tests
- Fully custom error messages
- User-defined ordering of the checks (`static_assert` doesn't do this)
- Early return and bailing out



Compile-time testing design decisions

- 1. Use expect-success testing**
- 2. Report compile-time errors at run-time**
- 3. All test code should compile, even the code that tests compile-time errors**



Early exit mechanism

```
TEST("...", "...") {  
    ASSERT(foo()); // "Fatal"  
    EXPECT(bar()); // "Non-fatal"  
}
```

If `foo()` is `false`, `bar` should not execute



Early exit mechanism

- `return` or `throw`?
- `return` only exits the current scope, while `throw` "bubbles up"
- With `return`, the checks can only be used in certain functions
- With `throw`, the checks cannot be used at compile-time pre C++26
- I chose `return`



Compile-time testing design decisions

1. Use expect-success testing
2. Report compile-time errors at run-time
3. All test code should compile, even the code that tests compile-time errors
4. Exit early from failed fatal checks using `return`



My compile-time-first unit testing library

My unit testing library **k3tchup**

- <https://github.com/k3DW/k3tchup>
- C++20 library, decoupled from parsing library **tok3n**
- Cross-platform (MSVC, Clang, GCC)
- API inspired by GoogleTest
- Functionality inspired by Snitch
- Prioritizes compile-time testing
- All errors are reported at run-time
- Still a work in progress...



k3tchup "regular" checks

- Basic usage
- Reusability for compile-time
- Keeping the full trace
- Dependent checks



Basic k3tchup - "regular" checks

```
#include <k3/k3tchup_main.hpp>
#include <optional>

FIXTURE("my fixture");

TEST("my fixture", "my test") {

    constexpr std::optional<int> o = 4;

    ASSERT_COMPILE_TIME(o.has_value()) << "optional did not have a value";
    EXPECT_COMPILE_TIME(*o == 5) << "optional's value was not 5, it was " << *o;

}
```

<https://github.com/k3DW/k3tchup>



Basic k3tchup - "regular" checks

```
Running fixture "my fixture"  
    Test "my test" - 2 checks / 1 error.  
Fixture "my fixture" - 1 test / 1 failure.
```

```
=====
```

```
1 test failed.
```

```
Fixture "my fixture"  
Test "my test" failed with 1 error.  
[Compile-time non-fatal error]  
    at path\to\test_file.cpp:11  
    optional's value was not 5, it was 4
```



Basic k3tchup - "regular" checks

```
#include <k3/k3tchup_main.hpp>
#include <optional>

FIXTURE("my fixture");

TEST("my fixture", "my test") {

    constexpr std::optional<int> o = 4;

    ASSERT_COMPILE_TIME(o.has_value()) << "optional did not have a value";
    EXPECT_COMPILE_TIME(*o == 5) << "optional's value was not 5, it was " << *o;

}
```



Basic k3tchup - "regular" checks

```
#include <k3/k3tchup_main.hpp>
#include <optional>

FIXTURE("my fixture");

TEST("my fixture", "my test") {

    constexpr std::optional<int> o = 4;

    ASSERT_RUN_TIME(o.has_value()) << "optional did not have a value";
    EXPECT_RUN_TIME(*o == 5) << "optional's value was not 5, it was " << *o;

}
```



Basic k3tchup - "regular" checks

```
#include <k3/k3tchup_main.hpp>
#include <optional>

FIXTURE("my fixture");

TEST("my fixture", "my test") {

    constexpr std::optional<int> o = 4;

    ASSERT_COMPILE_AND_RUN_TIME(o.has_value()) << "optional did not have a value";
    EXPECT_COMPILE_AND_RUN_TIME(*o == 5) << "optional's value was not 5, it was " << *o;

}
```



Basic k3tchup - "regular" checks

```
#include <k3/k3tchup_main.hpp>
#include <optional>

FIXTURE("my fixture");

TEST("my fixture", "my test") {

    constexpr std::optional<int> o = 4;

    ASSERT(o.has_value()) << "optional did not have a value";
    EXPECT(*o == 5) << "optional's value was not 5, it was " << *o;

}
```



Compile-time testing design decisions

1. Use expect-success testing
2. Report compile-time errors at run-time
3. All test code should compile, even the code that tests compile-time errors
4. Exit early from failed fatal checks using `return`
5. Ability to decide compile-time vs run-time at each check



Reusability for compile-time

```
TEST("my fixture", "my test") {  
  
    ASSERT(foo().has_value());  
    EXPECT(*foo() == 5);  
  
}
```



Reusability for compile-time



```
constexpr void check_option1(std::optional<int> o) {  
  
    ASSERT(o.has_value());  
    EXPECT(*o == 5);  
  
}  
  
TEST("my fixture", "test") {  
    check_option1(5);  
}
```

Reusability for compile-time

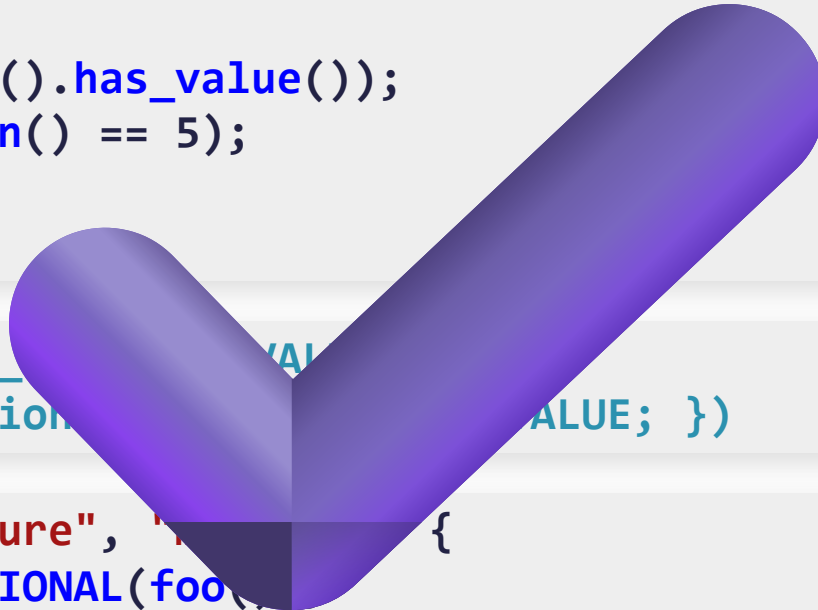
```
template <std::invocable<> Fn>
constexpr void check_optional(Fn fn) {

    ASSERT(fn().has_value());
    EXPECT(*fn() == 5);

}
```

```
#define CHECK_OPTIONAL(fn) \
    check_optional(fn, 5);
```

```
TEST("my fixture", "foo") {
    CHECK_OPTIONAL(foo);
}
```



Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)



Keeping the full trace

```
12 template <std::invocable<> Fn>
13 constexpr void check_optional(Fn fn) {
14     ASSERT(fn().has_value());
15     EXPECT(*fn() == 5);
16 }
```

```
23 TEST("my fixture", "my test") {
24     CHECK_OPTIONAL(std::optional<int>(3));
25     CHECK_OPTIONAL(std::optional<int>(4));
26 }
```



Keeping the full trace

```
Running fixture "my fixture"
  Test "my test" - 4 checks / 2 errors.
Fixture "my fixture" - 1 test / 1 failure.

=====

1 test failed.

Fixture "my fixture"
Test "my test" failed with 2 errors.
[Compile-time and run-time non-fatal error]
  at path\to\slideware.cpp:15
[Compile-time and run-time non-fatal error]
  at path\to\slideware.cpp:15
```



Keeping the full trace

- `ASSERT_THAT` and `EXPECT_THAT`
- Macros that take a "packet"
- "Packets" must be callable with no arguments
- Preserves the trace



Keeping the full trace

```
10 template <std::invocable<> Fn>
11 constexpr auto check_optional(Fn) {
12     return [] {
13         constexpr Fn fn;
14         ASSERT(fn().has_value());
15         EXPECT(*fn() == 5);
16     };
17 }
```

```
23 TEST("my fixture", "my test") {
24     EXPECT_THAT(CHECK_OPTIONAL(std::optional<int>(3)));
25     EXPECT_THAT(CHECK_OPTIONAL(std::optional<int>(4)));
26 }
```



Keeping the full trace

```
Running fixture "my fixture"
  Test "my test" - 4 checks / 2 errors.
Fixture "my fixture" - 1 test / 1 failure.

=====

1 test failed.

Fixture "my fixture"
Test "my test" failed with 2 errors.
[Compile-time and run-time non-fatal error]
  at path\to\slideware.cpp:24
  at path\to\slideware.cpp:15
[Compile-time and run-time non-fatal error]
  at path\to\slideware.cpp:25
  at path\to\slideware.cpp:15
```



Keeping the full trace

- The packet is only called at run-time
- The packet's body should handle the compile-time vs run-time checks
- These macros provide the primitive mechanism for matchers



Keeping the full trace

```
TEST("any_of_parser", "Requirements")
{
    EXPECT_THAT(the_parser<Single> | has_value_type<char>);

    EXPECT_THAT(the_parser<Single> | has_family<any_of_family>);

    EXPECT_THAT(the_parser<Single> | is_parser_for<char>.with_result<output_span<char>>);

    // ...
}
```



Keeping the full trace

```
#define EXPECT_THAT(PACKET) \
{ \
    context::trace_context ctx; \
    (PACKET)(); \
} static_assert(true, "require semicolon")
```



Keeping the full trace

- Run packets at run-time
- Therefore we can use RAI for `static` state



Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI



Keeping the full trace

```
#define EXPECT_THAT(PACKET) \
{ \
    context::trace_context ctx; \
    (PACKET)(); \
} static_assert(true, "require semicolon")
```



Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon



Dependent checks

```
constexpr std::optional<int> foo() {  
    return std::nullopt;  
}  
  
TEST("my fixture", "my test") {  
  
    ASSERT(foo().has_value());  
    EXPECT(*foo() == 5);  
}
```

Early return for run-time... but not compile-time



```

path\to\slideware.cpp(11,5): error C7595:
'test_impl_<221864>::_run::<lambda_2>::operator ()':
call to immediate function is not a constant expression [path\to\build\slideware.vcxproj]
C:\Program Files\...\include\optional(203,9):
failure was caused by call of undefined function or one not declared 'constexpr'
C:\Program Files\...\include\optional(203,9):
see usage of '_CrtDbgReport'
path\to\slideware.cpp(11,5):
the call stack of the evaluation (the oldest call first) is
    path\to\slideware.cpp(11,5):
    while evaluating function 'bool test_impl_<221864>::_run::<lambda_2>::operator ()<
        std::integral_constant<size_t,57049165933>>(_k3_k3tchup_type_parameter_) const'
    with
    [
        _k3_k3tchup_type_parameter_=std::integral_constant<size_t,57049165933>
    ]
    path\to\slideware.cpp(11,5):
    ...
...

```



Dependent checks

- `k3tchup` reports errors at run-time
- ...even compile-time errors
- How do we handle dependent checks?



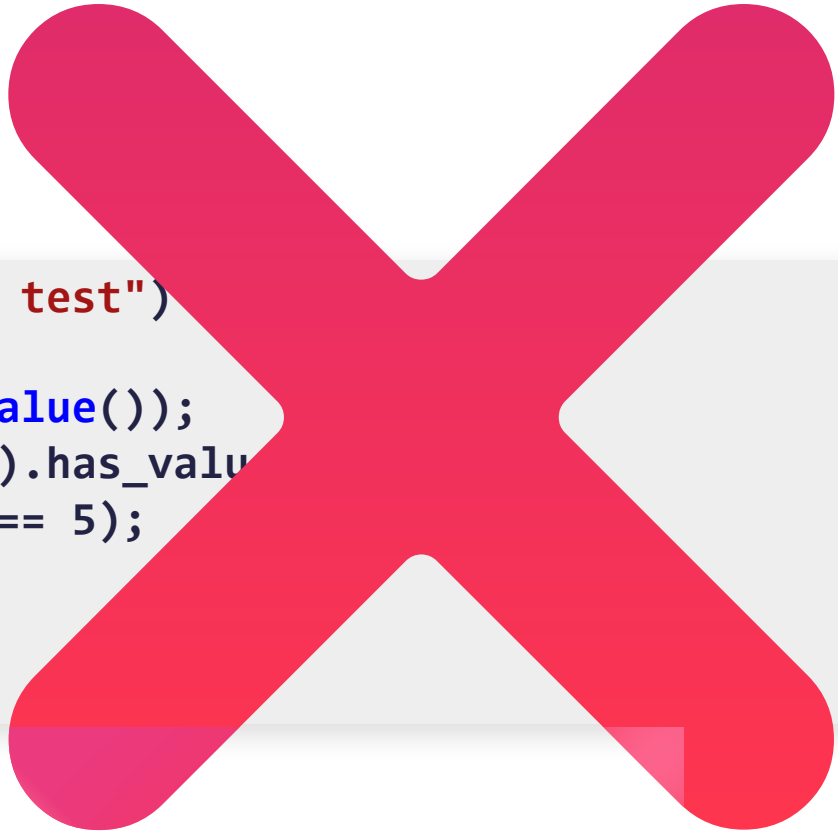
Dependent checks - what can we do?

- **Put the dependent check in a dependent scope**



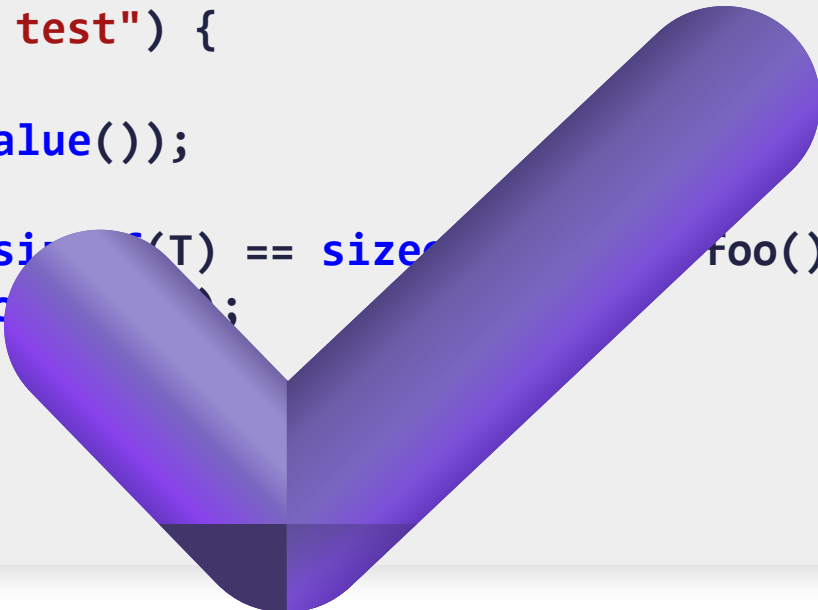
Dependent scope

```
TEST("my fixture", "my test")  
  
    ASSERT(foo().has_value());  
    if constexpr (foo().has_value())  
        EXPECT(*foo() == 5);  
}  
  
}
```



Dependent scope

```
TEST("my fixture", "my test") {  
  
    ASSERT(foo().has_value());  
    [<class T>(T) {  
        if constexpr (sizeof(T) == sizeof(int)) {  
            EXPECT(*foo());  
        }  
    }(&0);  
}
```



Dependent scope

```
TEST("my fixture", "my test") {  
  
    std::optional<int> o = 4;  
  
    ASSERT(o.has_value());  
    [&<class T>(T) {  
        if constexpr (sizeof(T) == sizeof(int) && o.has_value()) {  
            ASSERT(*o == 5);  
        }  
    }(&);  
  
    ASSERT(  
        something_importantDepending_on_o_being_5()  
    );  
}
```

The important check still runs!!



Dependent checks - what can we do?

- ~~Put the dependent check in a dependent scope~~
- Keep track of whether a fatal error has already occurred?



Keeping track at compile-time

```
constexpr std::optional<int> foo() {  
    return std::nullopt;  
}  
  
TEST("my fixture", "my test") {  
  
    ASSERT(foo().has_value());  
    EXPECT(*foo() == 5);  
}
```

Make this compile



Keeping track at compile-time

```
ASSERT(foo().has_value())
```

Expands to...

```
switch(0) case 0: default:
if (
    const auto _k3_k3tchup_1 = k3tchup::context::check(
        k3tchup::detail::eval_condition(foo().has_value()),
        foo().has_value()
    );
    _k3_k3tchup_1
) {}
else
    return k3tchup::void_assigner{} =
        k3tchup::context::add_error(_k3_k3tchup_1)
```



Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon
4. "Ambiguous else blocker" `switch(0) case 0: default:`



Keeping track at compile-time

```
ASSERT(foo().has_value())
```

Expands to...

```
switch(0) case 0: default:
if (
    const auto _k3_k3tchup_1 = k3tchup::context::check(
        k3tchup::detail::eval_condition(foo().has_value()),
        foo().has_value()
    );
    _k3_k3tchup_1
) {}
else
    return k3tchup::void_assigner{} =
        k3tchup::context::add_error(_k3_k3tchup_1)
```



Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon
4. "Ambiguous else blocker" `switch(0) case 0: default:`
5. "Void assignment helper" for message streaming




```

switch(0) case 0: default:
if (
    const auto _k3_k3tchup_1 = k3tchup::context::check(
        k3tchup::detail::eval_condition(foo().has_value()),
        foo().has_value()
    );
    _k3_k3tchup_1
) {}
else
    return k3tchup::void_assigner{} =
        k3tchup::context::add_error(_k3_k3tchup_1);

```

```

switch(0) case 0: default:
if (
    const auto _k3_k3tchup_1 = k3tchup::context::check(
        k3tchup::detail::eval_condition(*foo() == 5),
        *foo() == 5
    );
    _k3_k3tchup_1
) {}
else
    k3tchup::void_assigner{} =
        k3tchup::context::add_error(_k3_k3tchup_1);

```



Keeping track at compile-time

- If we encounter a compile-time fatal error...
- ...register this function as "errored"...
- ...using stateful metaprogramming



Friend injection

```
template <class T>
struct error_key {
    friend auto find(error_key<T>);
};

template <class T>
struct set_error_for {
    friend auto find(error_key<T>) { return 0; }
};

template <class T, class Unique = decltype([]{})>
concept has_error = requires {
    find(error_key<T>{});
    typename Unique;
};
```



Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon
4. "Ambiguous else blocker" `switch(0) case 0: default:`
5. "Void assignment helper" for message streaming
6. Lambda `[]{} in a default template parameter`



Friend injection

```
k3tchup::detail::eval_condition(foo().has_value())
```

Replaced with...

```
[&]<class T>(T) consteval {  
    if constexpr (k3tchup::detail::has_error<T>) {  
        return true;  
    } else if constexpr (foo().has_value()) {  
        return true;  
    } else {  
        static_assert(sizeof(k3tchup::detail::set_error_for<T>));  
        return false;  
    }  
}(unique_type_for_this_function{})
```



Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon
4. "Ambiguous else blocker" `switch(0) case 0: default:`
5. "Void assignment helper" for message streaming
6. Lambda `[]{} in a default template parameter`
7. Instantiate a class template with `sizeof`



Friend injection

```
k3tchup::detail::eval_condition(*foo() == 5)
```

Replaced with...

```
[&<class T>(T) consteval {  
    if constexpr (k3tchup::detail::has_error<T>) {  
        return true;  
    } else {  
        return *foo() == 5;  
    }  
}(unique_type_for_this_function{})
```



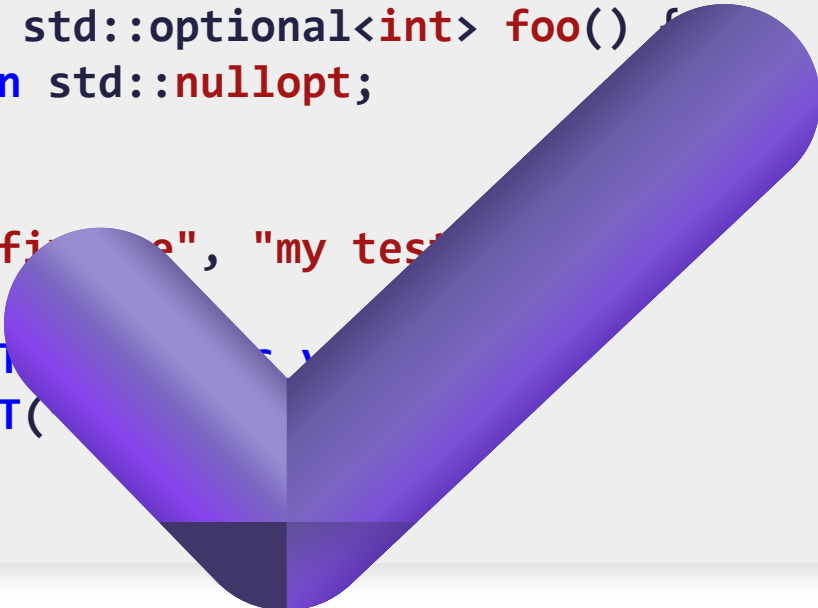
Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon
4. "Ambiguous else blocker" `switch(0) case 0: default:`
5. "Void assignment helper" for message streaming
6. Lambda `[]{} in a default template parameter`
7. Instantiate a class template with `sizeof`
8. Friend injection for stateful metaprogramming



Straight-line compile-time checks!

```
constexpr std::optional<int> foo() {  
    return std::nullopt;  
}  
  
TEST("my function", "my test") {  
    ASSERT_FALSE(foo());  
    EXPECT_EQ(foo(), std::nullopt);  
}
```



Something still missing...

```
constexpr std::optional<int> foo() {  
    // ...  
}  
  
TEST("my fixture", "something missing") {  
  
    RUN_THE_WHOLE_LAMBDA_CT_AND_RT([]{  
        auto o = foo();  
        ASSERT(foo().has_value());  
        EXPECT(*foo() == 5);  
    });  
}
```



Compile-time testing design decisions

1. Use expect-success testing
2. Report compile-time errors at run-time
3. All test code should compile, even the code that tests compile-time errors
4. Exit early from failed fatal checks using `return`
5. Ability to decide compile-time vs run-time at each check
6. Ability to decide compile-time vs run-time for a whole function



k3tchup "stateful" checks

```
TEST("my fixture", "my test") {  
  
    EXPECT_THAT([](k3::k3tchup::state& state) {  
        auto o = foo();  
        ASSERT(state, o.has_value()) << "optional did not have a value";  
        EXPECT(state, *o == 5) << "optional's value was not 5";  
    });  
  
}
```



k3tchup "stateful" checks

- New overload of `ASSERT_THAT` and `EXPECT_THAT`
- Convenience macros `ASSERT` and `EXPECT`
- A whole new type of test
- Accumulating errors at compile-time
- The "constexpr two-step"
- Merging the compile-time and run-time errors

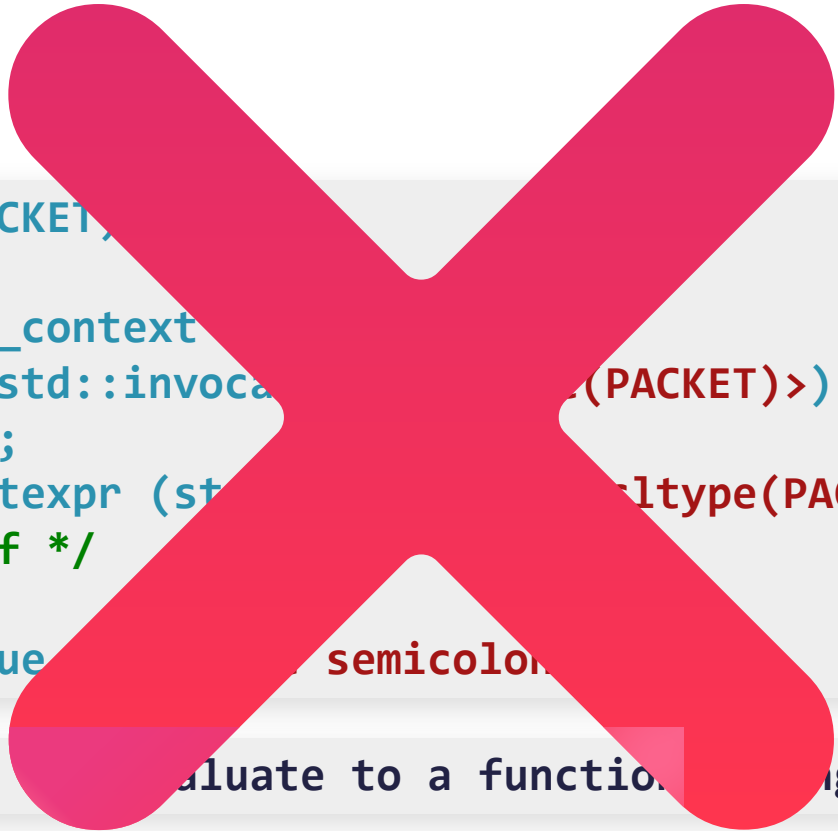


New overload of `ASSERT_THAT` and `EXPECT_THAT`

```
EXPECT_THAT([], () {  
    // ...  
});  
EXPECT_THAT([], (auto& state) {  
    // ...  
});
```



New overload of `ASSERT_THAT` and `EXPECT_THAT`

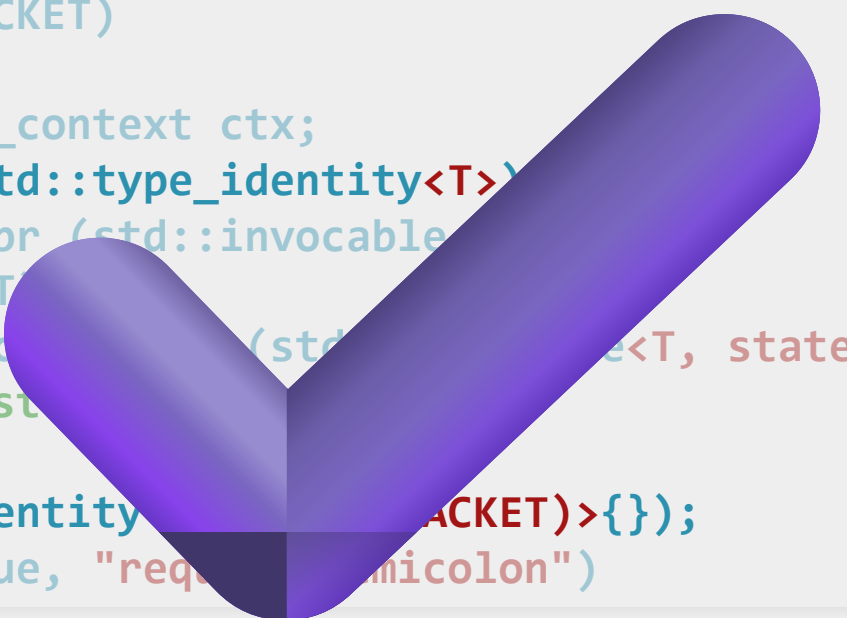


```
#define EXPECT_THAT(PACKET, ...) \
{ \
    context::trace_context \
    if constexpr (std::invocable(PACKET)>) { \
        (PACKET)(); \
    } else if constexpr (std::is_same_v<decltype(PACKET), state&>) { \
        /* do stuff */ \
    } \
} static_assert(true, "semicolon");
```

error C2064: term does not evaluate to a function taking 0 arguments

New overload of `ASSERT_THAT` and `EXPECT_THAT`

```
#define EXPECT_THAT(PACKET)
{
    context::trace_context ctx;
    [<class T>(std::type_identity<T>)
    if constexpr (std::invocable
        (PACKET, std::type_identity<T>)) {
    } else if constexpr (std::invocable
        (PACKET, std::type_identity<T>, state&)) {
        /* do st
    }
    }(std::type_identity<T>, PACKET)>{}));
} static_assert(true, "required semicolon")
```



Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon
4. "Ambiguous else blocker" `switch(0) case 0: default:`
5. "Void assignment helper" for message streaming
6. Lambda `[]{} in a default template parameter`
7. Instantiate a class template with `sizeof`
8. Friend injection for stateful metaprogramming
9. Add an extra template context when using if-constexpr dispatch



Convenience macros **ASSERT** and **EXPECT**

```
EXPECT_THAT([]() {  
    EXPECT(cond2); // Compile-time and run-time  
});
```

```
EXPECT_THAT([](auto& state) {  
    EXPECT(state, cond1); // Deduced  
});
```



Convenience macros **ASSERT** and **EXPECT**

- Overload based on arity
- If unary, use `{ASSERT|EXPECT}_COMPILE_AND_RUN_TIME`
- If binary, use `{ASSERT|EXPECT}_STATEFUL`



Convenience macros **ASSERT** and **EXPECT**

```
#define EXPECT(ARG0, ...) \
    SELECT( \
        IS_EMPTY(__VA_ARGS__), \
        EXPECT_COMPILE_AND_RUN_TIME, \
        EXPECT_STATEFUL \
    ) \
    (ARG0 __VA_OPT__(,) __VA_ARGS__)
```



Compile-time testing techniques

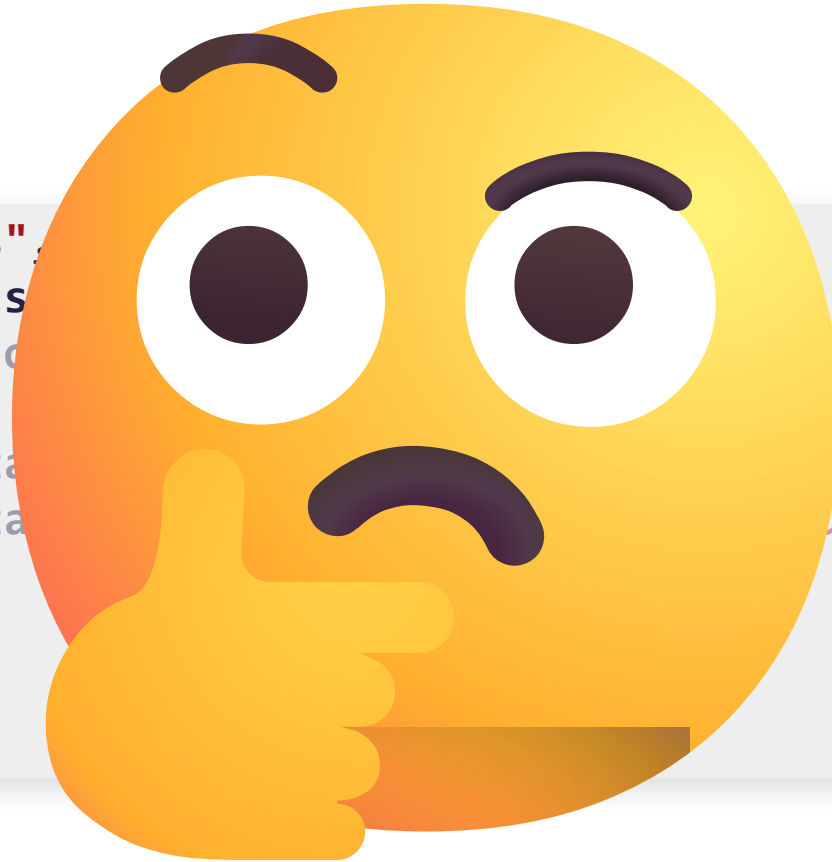
1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon
4. "Ambiguous else blocker" `switch(0) case 0: default:`
5. "Void assignment helper" for message streaming
6. Lambda `[]{} in a default template parameter`
7. Instantiate a class template with `sizeof`
8. Friend injection for stateful metaprogramming
9. Add an extra template context when using if-constexpr dispatch
10. Overload macros based on arity



A whole new type of test

```
TEST("vector of optionals"  
    EXPECT_THAT([], (auto& s  
        for (const auto& o  
            [&] {  
                ASSERT(sta  
                EXPECT(sta  
            }());  
        }  
    }));  
}
```

did not have a value";
ue was not 5";



A whole new type of test

```
TEST_CONSTEXPR("vector of optionals", "stateful checks", state) {  
    for (const auto& o : vec()) {  
        [&] {  
            ASSERT(state, o.has_value()) << "optional did not have a value";  
            EXPECT(state, *o == 5) << "optional's value was not 5";  
        }();  
    }  
}
```



Accumulating errors at compile-time

```
constexpr void lambda_body() {  
    if (not cond1) {  
        register_error(std::source_location::current());  
        return;  
    }  
    if (not cond2) {  
        register_error(std::source_location::current());  
    }  
}
```



```

class context {
    // ...
    static inline test_result* _current_result = nullptr;
};
class test_result_context {
public:
    test_result_context(test_result& result) {
        context::_current_result = &result;
    }
    ~test_result_context() {
        context::_current_result = nullptr;
    }
};

```

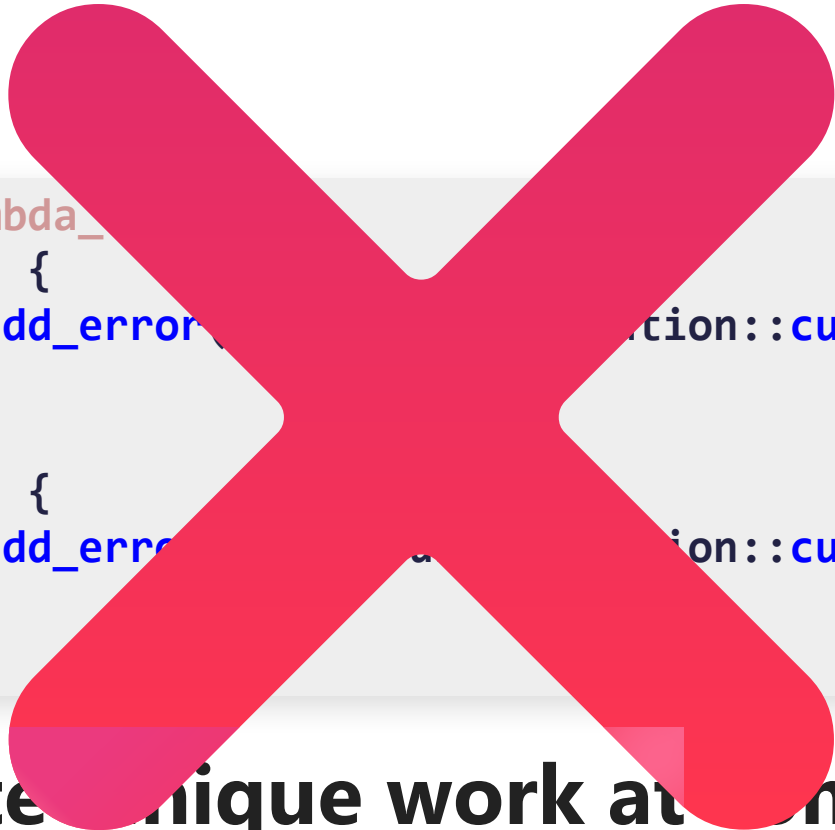
```

test_result test::run(std::ostream& os) const {
    test_result result(_name);
    {
        test_result_context ctx(result);
        _func();
    }
    result.print_brief(os);
    return result;
}

```



Accumulating errors at compile-time



```
constexpr void lambda_  
    if (not cond1) {  
        context::add_error(context::current());  
        return;  
    }  
    if (not cond2) {  
        context::add_error(context::current());  
    }  
}
```

Does this technique work at compile-time?

Accumulating errors at compile-time

```
// Simplified
constexpr void register_error() {
    static int count = 0;
    ++count;
}

constexpr void lambda_body() {
    if (not cond1) {
        register_error();
        return;
    }
    if (not cond2) {
        register_error();
    }
}
```



<https://godbolt.org/z/jK4W78qzc>

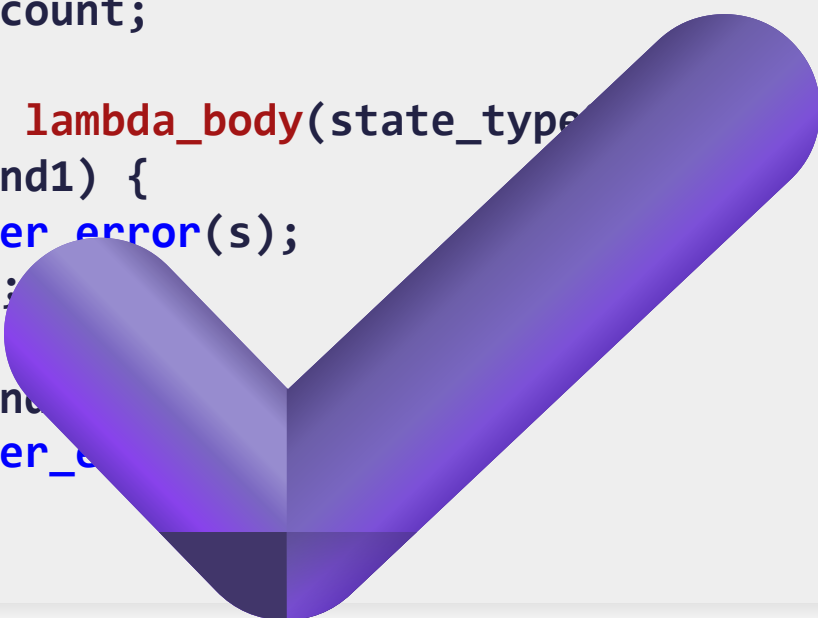
Accumulating errors at compile-time

- We need to accumulate the error somehow
- We can't access any mutable state external to the function
- Therefore we must pass in the state



Accumulating errors at compile-time

```
// Simplified
constexpr void register_error(state_type& s) {
    ++s.error_count;
}
constexpr void lambda_body(state_type& s) {
    if (not cond1) {
        register_error(s);
        return;
    }
    if (not cond2) {
        register_error(s);
    }
}
```



<https://godbolt.org/z/ooacoaWfP>

Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon
4. "Ambiguous else blocker" `switch(0) case 0: default:`
5. "Void assignment helper" for message streaming
6. Lambda `[]{} in a default template parameter`
7. Instantiate a class template with `sizeof`
8. Friend injection for stateful metaprogramming
9. Add an extra template context when using if-constexpr dispatch
10. Overload macros based on arity
11. Pass all state explicitly; cannot rely on global state at compile-time



The actual `state` type

```
class state {  
    friend struct detail::state_accessor;  
    constexpr state() = default;  
    std::size_t num_checks = 0;  
    std::optional<std::vector<detail::state_error>> errors{ std::in_place };  
};
```

It's all private!



Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon
4. "Ambiguous else blocker" `switch(0) case 0: default:`
5. "Void assignment helper" for message streaming
6. Lambda `[]{} in a default template parameter`
7. Instantiate a class template with `sizeof`
8. Friend injection for stateful metaprogramming
9. Add an extra template context when using if-constexpr dispatch
10. Overload macros based on arity
11. Pass all state explicitly; cannot rely on global state at compile-time
12. Use an "accessor" to keep a class entirely private



The "constexpr two-step"

```
constexpr std::string foo() {  
    return "abcdefghijklmnopqrstuvwxy";  
}  
static_assert(foo() == "abcdefghijklmnopqrstuvwxy");
```

```
int main() {  
    constexpr std::string str = foo();  
}
```

```
<source>:8:33: error: 'foo()' is not a constant expression  
because it refers to a result of 'operator new'
```

"Persistent allocations"

The "constexpr two-step"

```
constexpr std::string foo() {  
    return "abcdefghijklmnopqrstuvwxy";  
}
```

```
constexpr auto two_step_foo() {  
    constexpr std::size_t size = foo().size();  
    std::array<char, size> arr{};  
    std::ranges::copy(foo(), arr.begin());  
    return arr;  
}
```

```
int main() {  
    constexpr auto arr = two_step_foo();  
    std::string str(arr.data(), arr.size());  
}
```



The "constexpr two-step"

- Execute the constexpr function twice
- Once to get the size
- Once to get the data itself



Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon
4. "Ambiguous else blocker" `switch(0) case 0: default:`
5. "Void assignment helper" for message streaming
6. Lambda `[]{} in a default template parameter`
7. Instantiate a class template with `sizeof`
8. Friend injection for stateful metaprogramming
9. Add an extra template context when using if-constexpr dispatch
10. Overload macros based on arity
11. Pass all state explicitly; cannot rely on global state at compile-time
12. Use an "accessor" to keep a class entirely private
13. The "constexpr two-step"



The "three-step"?

```
const state ct_state = state_accessor::deserialize(
    [&]() consteval {
        constexpr auto size = [&]() consteval {
            state s = state_accessor::make();
            (PACKET)(s);
            return state_accessor::get_size(s);
        }();
        state s = state_accessor::make();
        (PACKET)(s);
        return state_accessor::serialize<size>(s);
    }()
);
```

```
state rt_state = state_accessor::make();
(PACKET)(rt_state);
```

```
state_accessor::report(ct_state, rt_state);
```



Merging the compile-time and run-time errors

```
EXPECT_THAT([], (auto& state) {  
    EXPECT(state, false);  
});
```

Without merging...

```
[Compile-time non-fatal error]  
    at path\to\slideware.cpp:42  
[Run-time non-fatal error]  
    at path\to\slideware.cpp:42
```

Merging the compile-time and run-time errors

- Compile-time error and run-time error are recorded separately
- If we want a nice output, must merge later
- The error contains: fatality, message, location
- Match them up at run-time, and report them together

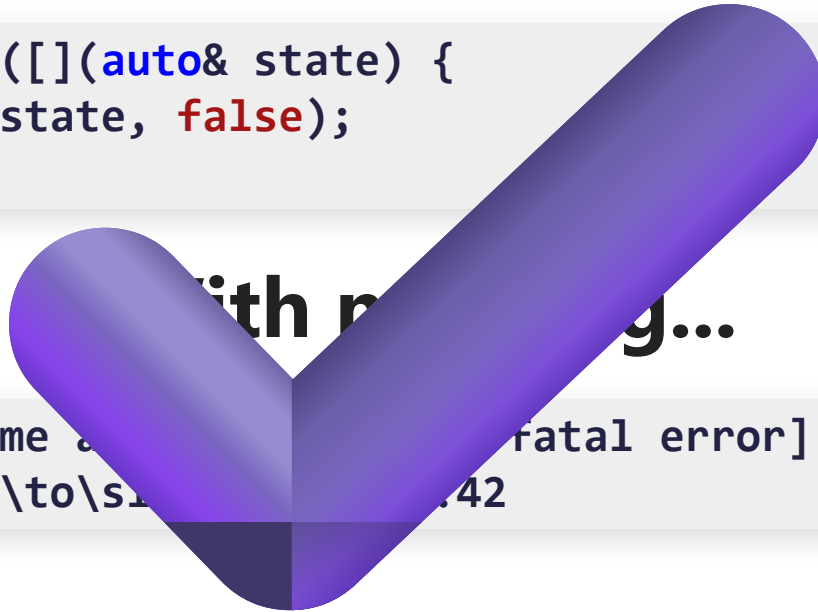


Merging the compile-time and run-time errors

```
EXPECT_THAT([], (auto& state) {  
    EXPECT(state, false);  
});
```

with merging...

```
[Compile-time error: fatal error]  
at path\to\src\main.cpp:42
```



Conclusion

Expand your toolbox

- Many specialized tools, but they may come in handy



Test your code!

- If you claim that your code runs at compile-time...
- ...test it at compile-time
- Your future self and your users will thank you



constexpr code is vast and complex

- But we can rein it in with compile-time testing
- We can more rigorously test for correct program behaviour
- Catch ill-formed programs early



k3tchup makes it easy

- **Straight-line code for compile-time conditions**
- **Equivalent to how you would do for run-time code**
- **Entire tests can be run at compile-time and run-time**
- **Scalable and ergonomic compile-time testing**



Compile-time testing design decisions

1. Use expect-success testing
2. Report compile-time errors at run-time
3. All test code should compile, even the code that tests compile-time errors
4. Exit early from failed fatal checks using `return`
5. Ability to decide compile-time vs run-time at each check
6. Ability to decide compile-time vs run-time for a whole function



Compile-time testing techniques

1. Lift a value into a lambda (must be captureless)
2. Use `static` with RAI
3. Use a `static_assert` to ensure a compound statement ends in a semicolon
4. "Ambiguous else blocker" `switch(0) case 0: default:`
5. "Void assignment helper" for message streaming
6. Lambda `[]{} in a default template parameter`
7. Instantiate a class template with `sizeof`
8. Friend injection for stateful metaprogramming
9. Add an extra template context when using if-constexpr dispatch
10. Overload macros based on arity
11. Pass all state explicitly; cannot rely on global state at compile-time
12. Use an "accessor" to keep a class entirely private
13. The "constexpr two-step"



k3tchup upcoming work

- Refining the identifier choices (e.g. `TEST_CONSTEXPR`)
- Integrating C++26 reflection
- Tests for whether an expression *can* run at compile-time
- ...and more!



k3tchup is a work in progress

- Follow along at <https://github.com/k3DW/k3tchup>
- Please come talk to me about `constexpr` testing



Thank you!

Braden Ganetsky

braden@ganets.ky

GitHub @k3DW

<https://ganets.ky/slides/2026/accuonsea>

